

AWS Certified Generative AI Developer — Professional

The Use Case

- Multinational financial services firm building an AI financial advisory system — market analysis, investment recommendations, advisor support
- Highly regulated: **SEC, FINRA**, and requirements differing across jurisdictions

Task	The question it answers
3.1	Is the content safe? (harmful output, hallucination)
3.2	Is the data safe? (who sees it, where it goes)
3.3	Can you prove it? (documentation, lineage, audit)

Governance isn't about the system behaving correctly — it's about being able to demonstrate to a regulator that it does.

Note: the transcript only covers compliance frameworks and data source tracking; organisational governance and continuous monitoring appear in the notes alone. Both are included here. This task also names specific APIs — worth recognising, though knowing *which service meets which governance need* matters more than memorising signatures.

1. Compliance Frameworks

SageMaker Model Cards

A model's combined birth certificate and report card.

Model purpose and intended use cases
Training data characteristics and limitations
Performance metrics across scenarios
Known limitations and potential risks

Financial additions:
SEC / FINRA compliance documentation
Fairness assessments across demographic groups
Explainability metrics for recommendations
Version control and approval workflows

```
create_model_card()
update_model_card()
export_model_card()
```

Why programmatic: manually written documentation goes stale — someone updates the model and forgets the card. Generate it in the **CI/CD pipeline** by extracting metrics from training jobs, processing jobs and endpoints, so the card is always current because no human writes it.

SageMaker ML Lineage Tracking

Answers: how was this model built — which data, which code, which parameters?

DAG structure:
Artifacts -> things (dataset, model, image)
Actions -> work (training job, processing job)
Contexts -> grouping (endpoint, experiment)
connected by Associations

```
create_artifact()
create_context()
create_action()
```

Why custom entities are needed: SageMaker captures lineage for its own jobs automatically, but anything done outside — an external data source, a manual step — must be linked in explicitly.

AWS Glue — data lineage

Type	Tracks
Model lineage	How the model was built
Data lineage	How the data was transformed

- Glue ETL jobs configured with **custom job parameters** to generate and update lineage metadata
- Provenance maintained through the **DynamicFrame** class and its methods
- Source-to-target mappings stored in the **Glue Data Catalog** — so "where did this column come from?" has an answer

Metadata tagging

```
TagResource API, applied consistently across services

DataSensitivity: confidential
RegulatoryScope: SEC
DataProvider:    Bloomberg
ModelVersion:    v3
Jurisdiction:    EU
```

- **Attribution** — provenance is visible throughout the ML lifecycle
- **Tag-based access control** — IAM policies can condition on tags
- Tags can be **enforced via SCPs and tag policies**, so untagged resources cannot be created

CloudWatch Logs — decision logging

```
Input parameters and decision factors
Confidence scores
ALTERNATIVE recommendations (the ones not chosen)
Timestamp and user context
```

Logging the **alternatives** matters because the regulator's question is "why did you recommend this *rather than that?*" Both options must be on record.

Audit trail requirements:

- End-to-end tracking from user query to final recommendation
- **Human interventions and overrides logged** — this is what demonstrates a person remained in control
- **Correlation IDs** linking related transactions — one recommendation may span five service calls
- Retention policies aligned to financial regulation

Structured logging (JSON, not plain text) makes logs queryable through Logs Insights, and **metric filters** turn log patterns into custom governance metrics.

2. Data Source Tracking

Where did the AI's answer come from? In finance this is not merely good practice — **licensing and copyright obligations** depend on it.

Glue Data Catalog — source registration

Crawlers with custom classifiers for specialised formats Incremental crawling strategies GetTables / GetDatabases for programmatic queries
Register every financial data source Classify by data type, provider, JURISDICTION Document licensing and usage rights Record data quality and reliability metrics

Licensing documentation is the non-obvious part — each provider imposes different terms on how their data may be used, and the catalog is where that lives.

CloudTrail configuration

Setting	Note
Multi-Region trails	Cover every region
Custom event selectors	Management events are on by default; DATA events are off by default because of volume — they must be enabled explicitly for compliance, or you won't know who read which S3 object
Athena queries	Trail logs live in S3; query them with Athena to produce compliance reports

Cross-account lineage

AddAssociation API + cross-account IAM roles + correct ARNs

Lineage continues across account boundaries — common when data lives in one account and training in another.

Data versioning and reproducibility

Option	Suited to
S3 versioning + lifecycle	Files, documents, raw data
SageMaker Feature Store time travel	Structured features — "what was this feature's value on 1 March 2024?"

Regulator: "reproduce this recommendation" -> same model version -> same DATA version -> the same result must appear

Without versioning this is impossible, because the data will have moved on.

Source attribution in AI outputs

- Track **which data sources influenced a specific recommendation**
- Confidence scores weighted by source reliability** — a recommendation built on a less reliable source should carry lower confidence
- Citation generation for regulatory documentation and client-facing transparency
- Compliance with copyright and licensing

Practical limit: you cannot say precisely which chunk contributed which sentence — but you can record **which chunks were in the prompt**, and that is the attribution.

Auditing the attribution itself

verification workflows for attribution ACCURACY periodic audits for attribution COMPLETENESS remediation processes for attribution GAPS

The attribution system is itself subject to audit — is it citing the right sources, and are any outputs escaping without attribution?

Lineage visualisation

Query the **SageMaker Lineage API** together with the Glue Data Catalog to generate lineage graphs. A regulator wants a picture of where data came from and where it went, not a log dump.

3. Organisational Governance Systems

Sections 1 and 2 concerned one model. This concerns **hundreds of models across many teams and accounts**.

Three layers of control

AWS Organizations level -> SCPs	
Account level -> IAM policies	
Service level -> resource policies	

Function	
IAM	Grants permission
SCP	Sets the ceiling on maximum permission — an account administrator cannot bypass it. SCPs never grant anything; they only limit.

"No account may deploy Bedrock outside us-east-1"
"Untagged resources cannot be created"
"Model customization jobs only in approved accounts"

Same idea as the permission boundary in Task 2.3, applied across the whole organisation.

Approval workflows — Step Functions

governance metrics check	
-> compliance checks	
-> required approvals (tracked in DynamoDB)	
-> promote or reject	
bias metrics within threshold?	-> continue
model card complete?	-> continue
evaluation passed?	-> continue
human approval received?	-> deploy

The clinical-validation and three-tier-approval patterns from Tasks 1.2 and 1.6, now driven by automated governance metrics.

SageMaker Model Dashboard

- Central view of model performance, compliance status and governance metrics across every model
- Extend with `create_monitoring_schedule()` and **EventBridge integration** for custom alerting
- With 200 models in production, somebody has to be able to see which are drifting and which are out of compliance

Role separation

development / evaluation / deployment / monitoring
-> separate responsibilities

The person who builds a model cannot approve it. Classic separation of duties.

SageMaker Clarify — bias and explainability

Metric	Measures
DPPL Difference in Positive Proportions in Labels	The gap in positive outcomes between two groups
DI Disparate Impact	The ratio of one group's positive rate to another's
DCR Difference in Conditional Rejection	The gap in rejection rates, conditioned on qualification

Stage	Question
Pre-training bias	Is the bias in the data ?
Post-training bias	Is the bias in the model's predictions ?

Both are needed — clean data can still produce a biased model, and a biased dataset may not always be amplified. In financial services, demographic bias in lending or investment recommendations is **illegal**, not merely undesirable.

Governance metrics and scale

% models with complete model cards
% models within bias thresholds
days since last evaluation
count of untagged resources

Tracked as CloudWatch custom metrics and visualised on dashboards. **Governance that isn't measured is just a policy document.** At scale — hundreds of models — enforcement must be automatic: service quotas, tagging strategies, automated checks. Cross-region consistency comes from Organizations, SCPs and centralised logging.

4. Continuous Monitoring and Advanced Controls

SageMaker Model Monitor

```
constraint suggestion algorithms -> build a baseline from training data
custom monitoring analysis code -> your own container and logic
```

```
Four monitoring types:
  data quality drift
  model quality drift
  bias drift
  feature attribution drift
```

Bias drift with Clarify

A model that was fair at deployment may not be six months later — the model is unchanged, but the applicant population has shifted.

```
Jensen-Shannon divergence
  baseline distribution (at training)
  current distribution (now)
-> divergence above threshold -> alert
```

Threshold tuning matters: too low produces false alarms, too high misses real drift.

Automated remediation

```
monitoring alarm fires
-> EventBridge rule
-> Lambda or Step Functions
-> automated model rollback
    or traffic shifting
    or alert and escalate
```

The blue/green rollback from Task 1.2, triggered automatically. **A bias violation cannot wait for someone to read an email.**

Token-level redaction

```
custom PRE-processing handler -> redact PII from input
-> model
custom POST-processing handler -> redact PII from output
```

Techniques: **pattern matching** (regex, known formats) or **NER models** (semantic). Implemented as custom handlers on inference endpoints.

```
"Your account 4532-1234 holds $5,000"
-> "Your account [REDACTED] holds $5,000"
```

"Token-level" means the response is not blocked — only the sensitive tokens are removed, so the rest of the answer remains useful.

Output policy filters

Bedrock Guardrails or custom mechanisms enforce content policies and organisational guidelines. Where Guardrails is insufficient, the notes suggest **prompt engineering, RAG with trusted sources, and output filtering** — Guardrails is one tool, not the entire solution.

Automated compliance testing

```
safety / security / compliance benchmarks
-> evaluated continuously
-> ONGOING regulatory readiness
```

Compliance is not a certificate that was once obtained; it is a state that must be continuously maintained. Same principle as the adversarial testing framework in Task 3.1.

Misuse monitoring

```
anomaly detection on:
  usage patterns
  content generation requests
  system interactions

watch for: one user suddenly at 100x volume,
            unusual recommendation patterns,
            off-hours access, systematic probing
```

Automated compliance reporting

```
extract metrics from CloudWatch
-> compile governance data
-> generate reports with QuickSight
```

Producing a quarterly regulatory report by hand takes weeks; an automated pipeline keeps it ready every day.

The governance loop

```
Deploy
-> Model Monitor (data / model / bias / attribution drift)
-> violation -> EventBridge -> Lambda or Step Functions
-> automated remediation (rollback or traffic shift)
-> CloudWatch metrics
-> QuickSight compliance reports
-> model card updated
-> repeat
```

Service limits and feature availability change. Verify specific numbers against current AWS documentation before the exam.